



OP Stack Customization

- Code Changes on the optimism Source for Guru Chain

2026.09

Gurufin Research Center

조광현

Tillo Tulkinbekov

Table of Contents

Overview	2
Document information	2
Summary of customizations	2
op-deployer	3
Why	3
Old version (current)	3
New version (customization)	8
New environment variables	8
Dynamic chain support assignment	8
op-challenger (and related)	12
Why	12
Old version	12
New version	12
Affected modules	12

Overview

This document contains the full documentation of the code changes made on the optimism source code to custom-build it for Guru Chain.

Upstream OP Stack is written for Ethereum mainnet and Sepolia. Chain specific data such as contract addresses, roles and standard versions is hardcoded for those two chains, and several components assume go-ethereum block identity semantics. Both assumptions have to be relaxed before the stack can be deployed on Guru Chain. The two components below are the ones that needed source changes.

Document information

Item	Value
Title	OP Stack Customization
Category	Research
Author	Tillo Tulkinbekov
Created	August 13, 2026
Last updated	September 15, 2026
Target	optimism (OP Stack) source code
Purpose	Custom build for Guru Chain as L1

Summary of customizations

No	Component	Goal	Affected modules
1	op-deployer	Register a custom L1 chain and its CREATE2 deployer at runtime	op-deployer
2	op-challenger	Pin blocks by block number instead of block hash	op-challenger, op-chain-ops, op-dispute-mon, op-e2e

op-deployer

Why

- The optimism source is highly customized for the Ethereum and Sepolia chains, so all chain related data such as addresses and config values is hardcoded for these chains.
- Deploying the OP Stack on a different L1 therefore requires code updates to register the new chain information.
- We plan to make this approach dynamic, so that the chain information can be registered at runtime and an L2 can be deployed on any L1 chain with EVM support.



Goals:

1. register a custom chain ID for op-deployer operations
2. runtime registration of the CREATE2 address

Old version (current)

All config parameters are hardcoded. Every lookup is a switch over chain ID 1 (mainnet) and 11155111 (sepolia), and any other chain ID is rejected as unsupported:

```
package standard

import (
    ...
)

const (
    GasLimit          uint64 = 60_000_000
    BasefeeScalar     uint32 = 1368
    BlobBaseFeeScalar uint32 = 801949
    WithdrawalDelaySeconds uint64 = 302400
    MinProposalSizeBytes uint64 = 126000
    ChallengePeriodSeconds uint64 = 86400
    ProofMaturityDelaySeconds uint64 = 604800
    DisputeGameFinalityDelaySeconds uint64 = 302400
    MIPSVersion        uint64 = 7
    DisputeGameType     uint32 = 1 // PERMISSIONED game type
    DisputeMaxGameDepth uint64 = 73
    DisputeSplitDepth   uint64 = 30
    DisputeClockExtension uint64 = 10800
    DisputeMaxClockDuration uint64 = 302400
    Eip1559DenominatorCanyon uint64 = 250
)
```

```

Eip1559Denominator          uint64 = 50
Eip1559Elasticity           uint64 = 6

ContractsV160Tag            = "op-contracts/v1.6.0"
ContractsV180Tag            = "op-contracts/v1.8.0-rc.4"
ContractsV170Beta1L2Tag     = "op-contracts/v1.7.0-beta.1+l2-contracts"
ContractsV200Tag            = "op-contracts/v2.0.0"
ContractsV300Tag            = "op-contracts/v3.0.0"
ContractsV400Tag            = "op-contracts/v4.0.0-rc.7"
ContractsV410Tag            = "op-contracts/v4.1.0"
CurrentTag                  = ContractsV410Tag
)

var DisputeAbsolutePrestate =
common.HexToHash("0x038512e02c4c3f7bdaec27d00edf55b7155e0905301e1a88083e4e0a6764d54c"
)

var VaultMinWithdrawalAmount = mustHexBigFromHex("0x8ac7230489e80000")

var GovernanceTokenOwner =
common.HexToAddress("0xDeaDDEaDDeAdDeAdDEAdDEaddeAddEAdDEAdDEad")

func L1VersionsFor(chainID uint64) (validation.Versions, error) {
    switch chainID {
    case 1:
        return validation.StandardVersionsMainnet, nil
    case 11155111:
        return validation.StandardVersionsSepolia, nil
    default:
        return nil, fmt.Errorf("unsupported chain ID: %d", chainID)
    }
}

func GuardianAddressFor(chainID uint64) (common.Address, error) {
    switch chainID {
    case 1:
        return common.Address(validation.StandardConfigRolesMainnet.Guardian), nil
    case 11155111:
        return common.Address(validation.StandardConfigRolesSepolia.Guardian), nil
    default:
        return common.Address{}, fmt.Errorf("unsupported chain ID: %d", chainID)
    }
}

```

```

func ChallengerAddressFor(chainID uint64) (common.Address, error) {
    switch chainID {
    case 1:
        return common.Address(validation.StandardConfigRolesMainnet.Challenger), nil
    case 11155111:
        return common.Address(validation.StandardConfigRolesSepolia.Challenger), nil
    default:
        return common.Address{}, fmt.Errorf("unsupported chain ID: %d", chainID)
    }
}

func SuperchainFor(chainID uint64) (superchain.Superchain, error) {
    switch chainID {
    case 1:
        return superchain.GetSuperchain("mainnet")
    case 11155111:
        return superchain.GetSuperchain("sepolia")
    default:
        return superchain.Superchain{}, fmt.Errorf("unsupported chain ID: %d",
chainID)
    }
}

func OPCMImplAddressFor(chainID uint64, tag string) (common.Address, error) {
    versionsData, err := L1VersionsFor(chainID)
    if err != nil {
        return common.Address{}, fmt.Errorf("unsupported chainID: %d", chainID)
    }
    versionData, ok := versionsData[validation.Semver(tag)]
    if !ok {
        return common.Address{}, fmt.Errorf("unsupported tag for chainID %d: %s",
chainID, tag)
    }
    if versionData.OPContractsManager.Address != nil {
        // op-contracts/v1.8.0 and earlier use proxied opcm
        return common.Address(*versionData.OPContractsManager.Address), nil
    }
    if versionData.OPContractsManager.ImplementationAddress != nil {
        // op-contracts/v2.0.0-rc.1 and later use non-proxied opcm
        return common.Address(*versionData.OPContractsManager.ImplementationAddress),
nil
    }
    return common.Address{}, fmt.Errorf("OPContractsManager address is nil for tag
%s", tag)
}

```

```

}

// SuperchainProxyAdminAddrFor returns the address of the Superchain ProxyAdmin for
// the given chain ID.
// These have been verified to be the ProxyAdmin addresses on Mainnet and Sepolia.
// DO NOT MODIFY THIS METHOD WITHOUT CLEARING IT WITH THE EVM SAFETY TEAM.
func SuperchainProxyAdminAddrFor(chainID uint64) (common.Address, error) {
    switch chainID {
    case 1:
        return common.HexToAddress("0x543bA4AADBAb8f9025686Bd03993043599c6fB04"), nil
    case 11155111:
        return common.HexToAddress("0x189aBAAaa82DfC015A588A7dbaD6F13b1D3485Bc"), nil
    default:
        return common.Address{}, fmt.Errorf("unsupported chain ID: %d", chainID)
    }
}

func L1ProxyAdminOwner(chainID uint64) (common.Address, error) {
    switch chainID {
    case 1:
        return
common.Address(validation.StandardConfigRolesMainnet.L1ProxyAdminOwner), nil
    case 11155111:
        return
common.Address(validation.StandardConfigRolesSepolia.L1ProxyAdminOwner), nil
    default:
        return common.Address{}, fmt.Errorf("unsupported chain ID: %d", chainID)
    }
}

func L2ProxyAdminOwner(chainID uint64) (common.Address, error) {
    switch chainID {
    case 1:
        return
common.Address(validation.StandardConfigRolesMainnet.L2ProxyAdminOwner), nil
    case 11155111:
        return
common.Address(validation.StandardConfigRolesSepolia.L2ProxyAdminOwner), nil
    default:
        return common.Address{}, fmt.Errorf("unsupported chain ID: %d", chainID)
    }
}

func ProtocolVersionsOwner(chainID uint64) (common.Address, error) {

```

```

    switch chainID {
    case 1:
        return
    common.Address(validation.StandardConfigRolesMainnet.ProtocolVersionsOwner), nil
    case 11155111:
        return
    common.Address(validation.StandardConfigRolesSepolia.ProtocolVersionsOwner), nil
    default:
        return common.Address{}, fmt.Errorf("unsupported chain ID: %d", chainID)
    }
}

// DefaultHardforkScheduleForTag is used to determine which hardforks should be
// activated by default given a
// contract tag. For example, passing in v1.6.0 will return all hardforks up to and
// including Granite. This allows
// OP Deployer to set sane defaults for hardforks. This is not an ideal solution, but
// it will have to work until we get
// to MCP L2.
func DefaultHardforkScheduleForTag(tag string) *genesis.UpgradeScheduleDeployConfig {
    sched := &genesis.UpgradeScheduleDeployConfig{
        L2GenesisRegolithTimeOffset: op_service.U64UtilPtr(0),
        L2GenesisCanyonTimeOffset:   op_service.U64UtilPtr(0),
        L2GenesisDeltaTimeOffset:    op_service.U64UtilPtr(0),
        L2GenesisEcotoneTimeOffset:  op_service.U64UtilPtr(0),
        L2GenesisFjordTimeOffset:    op_service.U64UtilPtr(0),
        L2GenesisGraniteTimeOffset:  op_service.U64UtilPtr(0),
    }

    switch tag {
    case ContractsV160Tag, ContractsV170Beta1L2Tag:
        return sched
    case ContractsV180Tag, ContractsV200Tag, ContractsV300Tag:
        sched.ActivateForkAtGenesis(rollup.Holocene)
    case ContractsV400Tag, ContractsV410Tag:
        sched.ActivateForkAtGenesis(rollup.Holocene)
        sched.ActivateForkAtGenesis(rollup.Isthmus)
    default:
        sched.ActivateForkAtGenesis(rollup.Holocene)
        sched.ActivateForkAtGenesis(rollup.Isthmus)
    }

    return sched
}

```

```
func mustHexBigFromHex(hex string) *hexutil.Big {
    num := hexutil.MustDecodeBig(hex)
    hexBig := hexutil.Big(*num)
    return &hexBig
}
```

New version (customization)

New environment variables

The custom chain is described entirely through the environment, so no rebuild is needed to target a different L1:

```
CUSTOM_CHAIN_ID
CUSTOM_CHAIN_NAME
CUSTOM_CHAIN_OPCM_IMPL
CUSTOM_CHAIN_OPCM_TAG
CUSTOM_CHAIN_GUARDIAN
CUSTOM_CHAIN_CHALLENGER
CUSTOM_CHAIN_L1_PROXY_ADMIN_OWNER
CUSTOM_CHAIN_L2_PROXY_ADMIN_OWNER
CUSTOM_CHAIN_PROTOCOL_VERSIONS_OWNER
CUSTOM_CHAIN_SUPERCHAIN_PROXY_ADMIN
CUSTOM_CHAIN_EXPLORER_API_URL

CUSTOM_CHAIN_CREATE2_DEPLOYER
```

Dynamic chain support assignment

The hardcoded switches are replaced by a profile map. The two built-in profiles are kept exactly as they are, and the custom profile is read from the environment and added to the map. A custom chain ID that shadows a built-in chain is rejected:

```
// L1Profiles returns the L1 profiles available to this process.
func L1Profiles() (map[uint64]L1Profile, error) {
    profiles := map[uint64]L1Profile{
        1: {
            Name:           "mainnet",
            ChainID:        1,
            SuperchainName: "mainnet",
            Versions:       validation.StandardVersionsMainnet,
            Roles:           validation.StandardConfigRolesMainnet,
```

```

// DO NOT MODIFY THIS ADDRESS WITHOUT CLEARING IT WITH THE EVM SAFETY
TEAM.

    SuperchainProxyAdmin:
common.HexToAddress("0x543bA4AADBAb8f9025686Bd03993043599c6fB04"),
    ExplorerAPIURL:      "https://api.etherscan.io/api",
    Create2Deployer:     script.DefaultDeterministicDeployerAddress,
},
11155111: {
    Name:                 "sepolia",
    ChainID:              11155111,
    SuperchainName:       "sepolia",
    Versions:             validation.StandardVersionsSepolia,
    Roles:                validation.StandardConfigRolesSepolia,
// DO NOT MODIFY THIS ADDRESS WITHOUT CLEARING IT WITH THE EVM SAFETY
TEAM.

    SuperchainProxyAdmin:
common.HexToAddress("0x189aBAAaa82DfC015A588A7dbaD6F13b1D3485Bc"),
    ExplorerAPIURL:      "https://api-sepolia.etherscan.io/api",
    Create2Deployer:     script.DefaultDeterministicDeployerAddress,
},
}

custom, err := customL1Profile()
if err != nil {
    return nil, fmt.Errorf("invalid custom L1 chain configuration: %w", err)
}
if custom != nil {
    if _, ok := profiles[custom.ChainID]; ok {
        return nil, fmt.Errorf("invalid custom L1 chain configuration: %s=%d
shadows a built-in chain", CustomChainIDEnvVar, custom.ChainID)
    }
    profiles[custom.ChainID] = *custom
}

return profiles, nil
}

// customL1Profile builds the custom L1 profile from the environment.
func customL1Profile() (*L1Profile, error) {
    chainID, err := envUint64(CustomChainIDEnvVar)
    if err != nil {
        return nil, err
    }
    if chainID == 0 {

```

```

        return nil, nil
    }

    name := strings.TrimSpace(os.Getenv(CustomChainNameEnvVar))
    if name == "" {
        name = fmt.Sprintf("custom-%d", chainID)
    }

    var roles validation.RolesConfig
    roleVars := []struct {
        envVar string
        dst     *validation.Address
    }{
        {CustomChainGuardianEnvVar, &roles.Guardian},
        {CustomChainChallengerEnvVar, &roles.Challenger},
        {CustomChainL1ProxyAdminOwnerEnvVar, &roles.L1ProxyAdminOwner},
        {CustomChainL2ProxyAdminOwnerEnvVar, &roles.L2ProxyAdminOwner},
        {CustomChainProtocolVersionsOwnerVar, &roles.ProtocolVersionsOwner},
    }
    for _, roleVar := range roleVars {
        addr, err := envAddress(roleVar.envVar)
        if err != nil {
            return nil, err
        }
        *roleVar.dst = validation.Address(addr)
    }

    superchainProxyAdmin, err := envAddress(CustomChainSuperchainProxyAdminVar)
    if err != nil {
        return nil, err
    }

    tag := strings.TrimSpace(os.Getenv(CustomChainOPCMTagEnvVar))
    if tag == "" {
        tag = CurrentTag
    }
    opcmImpl, err := envAddress(CustomChainOPCMImplEnvVar)
    if err != nil {
        return nil, err
    }
    // The version config is always registered so that the tag itself is considered
    // supported.
    // A missing OPCM address is reported when something actually asks for it, see
    // OPCMImplAddressFor.

```

```

versionConfig := validation.VersionConfig{}
if opcmImpl != (common.Address{}) {
    implAddr := validation.Address(opcmImpl)
    versionConfig.OPContractsManager = &validation.ContractData{
        Version:          tag,
        ImplementationAddress: &implAddr,
    }
}

return &L1Profile{
    Name:          name,
    ChainID:       chainID,
    Versions:      validation.Versions{validation.Semver(tag):
versionConfig},
    Roles:         roles,
    SuperchainProxyAdmin: superchainProxyAdmin,
    ExplorerAPIURL:
strings.TrimSpace(os.Getenv(CustomChainExplorerAPIURLEnvVar)),
    }, nil
}

```

op-challenger (and related)

Why

- op-challenger is responsible for watching game disputes on the L1 smart contract and validating them, reporting a false proposal when needed.
- As its purpose suggests, it continuously queries the L1 blocks for game dispute data.
- For querying the block information, Ethereum and Gurud have a mismatch:
 - Ethereum does not store the block hash in the header, it is recomputed every time.
 - Gurud stores the block hash computed by CometBFT.
- When a block is queried, Gurud expects the hash computed by CometBFT, `Header.Hash()`.
- op-challenger uses the Ethereum style recomputed hash, `keccak(rlp(header))`.
- The hashes do not match, so op-challenger cannot query the block data.
- We plan to make op-challenger use the block number instead of the hash, which works for both Gurud and Ethereum chains.



Goals:

1. pinning blocks by block number instead of block hash

Old version

```
// there are lots of usage of:
```

```
rpcblock.ByHash(block
```

New version

```
// all changed to:
```

```
rpcblock.ByNumber(head.Number.Uint64())
```

Affected modules

- optimism/op-challenger
- optimism/op-chain-ops
- optimism/op-dispute-mon
- optimism/op-e2e